

Updated Development Specification — Story #1: Outreach Message Guidance

User Story

As a student job seeker, I want help drafting my first LinkedIn message so that I feel confident reaching out to professionals without sounding awkward.

GitHub Issue: <https://github.com/dhy-ani/linkedin-clone/issues/2>

Architecture Overview

This story runs inside a single Flask process (app.py). The API gateway receives a POST /api/outreach/generate request, authenticates the caller via `_auth_user()`, fetches sender and recipient rows from database.py, then delegates draft generation to the stateless outreach.py module. The outreach module builds a draft from tone/goal templates and passes it through `_call_ai()`, which is a passthrough stub in P4 and will be replaced with an AWS Bedrock boto3 call in P5 with no schema changes elsewhere. The response is returned as JSON.

Sequence Diagram

Story #1 — Request Sequence

(Render at <https://mermaid.live> — paste block below)

```
sequenceDiagram
    participant Browser
    participant Flask as Flask app.py
    participant Outreach as outreach.py
    participant AI as AI Layer _call_ai
    participant DB as database.py

    Browser->>Flask: POST /api/outreach/generate
    Note over Flask: _auth_user() validates Bearer token
    Flask->>DB: get_user_by_id(sender_id)
    DB-->>Flask: sender dict
    Flask->>DB: get_user_by_id(recipient_id)
    DB-->>Flask: recipient dict
    Flask->>Outreach: sanitize_text(custom_note, 200)
    Outreach-->>Flask: sanitized string
    Flask->>Outreach: generate_outreach_message(sender, recipient, context)
    Outreach->>AI: _call_ai(prompt)
    AI-->>Outreach: draft string (P4: template; P5: Bedrock)
    Outreach-->>Flask: {draft, char_count, tone, tips[3], alternatives[2]}
    Flask-->>Browser: 200 JSON
```

Component Architecture Diagram

Story #1 — Component Architecture (classDiagram)

(Render at <https://mermaid.live> — paste block below)

```
classDiagram
    class AppModule {
        +outreach_generate() POST /api/outreach/generate
        -_auth_user() dict
    }
    class OutreachModule {
```

```

+VALID_TONES set
+VALID_GOALS set
+MAX_CUSTOM_NOTE int
+MAX_DRAFT_CHARS int
+sanitize_text(value, max_length) str
+generate_outreach_message(sender, recipient, context) dict
- _TONE_OPENERS dict
- _GOAL_LINES dict
- _TIPS dict
- _call_ai(prompt) str
- _first_name(full_name) str
}
class DatabaseModule {
+get_user_by_id(user_id) dict
+get_current_user(user_id) dict
- _DB_PATH str
- _sessions dict
- _connect() connection
}
AppModule --> OutreachModule : calls generate_outreach_message
AppModule --> DatabaseModule : calls get_user_by_id

```

Technology Stack

Layer	Technology	Version	Role
Frontend	HTML / CSS / Vanilla JS	ES2020	Outreach modal, api.js HTTP calls
Backend	Python + Flask	3.10+ / Flask 3.0	REST API gateway, request validation
Business Logic	outreach.py	—	Stateless draft generation and sanitization
Data Layer	database.py + SQLite WAL	—	User persistence, session tokens
AI Mock (P4)	_call_ai() passthrough	—	Returns formatted template string
AI Real (P5)	AWS Bedrock (Claude Haiku)	—	Swap in _call_ai() only — no other changes

API

POST /api/outreach/generate

Request body:

```
{ "recipient_id": 3, "tone": "professional", "goal": "job_inquiry",
  "custom_note": "I saw your post about..." }
```

Success response 200:

```
{ "draft": "Hi Jane, ...", "char_count": 312, "tone": "professional",
  "tips": ["...", "...", "..."], "alternatives": ["...", "..."] }
```

Error responses:

Code	Condition
400	Missing required field (recipient_id, tone, or goal)
401	No valid Bearer token and no demo fallback
404	recipient_id not found in database

Data Schemas

No new tables are created by this story. Read-only access to:

- users table — id, name, data JSON blob (experience, headline, skills, about, education, connections, openToWork, website, avatarColor)
- sessions table — token → user_id lookup via get_session_user_id()

State Diagram — Draft Lifecycle

Story #1 — Draft State

(Render at <https://mermaid.live> — paste block below)

```
stateDiagram-v2
    [*] --> Idle : Page load
    Idle --> Composing : User opens Outreach modal
    Composing --> Generating : User clicks Generate
    Generating --> DraftReady : 200 response received
    Generating --> Error : 400 or 404 or 500
    Error --> Composing : User corrects input
    DraftReady --> Copied : User clicks Copy to Clipboard
    Copied --> [*]
```

Flow Chart — generate_outreach_message()

Story #1 — Backend Flow

(Render at <https://mermaid.live> — paste block below)

```
flowchart TD
    A[POST /api/outreach/generate] --> B[_auth_user: validate Bearer token]
    B --> C{Token valid?}
    C -- No --> D[Fall back to demo user id=1]
    C -- Yes --> E[get_user_by_id sender]
    D --> E
    E --> F[get_user_by_id recipient]
    F --> G{Recipient found?}
    G -- No --> H[Return 404]
    G -- Yes --> I[sanitize_text custom_note 200]
    I --> J[Validate tone in VALID_TONES]
    J --> K{Valid?}
    K -- No --> L[Normalize to professional]
    K -- Yes --> M[generate_outreach_message]
    L --> M
    M --> N[_call_ai prompt]
    N --> O[Assemble response dict]
    O --> P[Return 200 JSON]
```

Security and Privacy

- Bearer tokens validated on every request via `_auth_user()`; fallback to demo user `id=1` applies in development only.
- All user-supplied strings pass through `sanitize_text()` before processing: HTML stripped, control characters removed, truncated to `max_length`.
- Passwords stored as SHA-256 hashes; `pw_hash` is never included in returned user dicts.
- No draft text is persisted in P4; the generated draft is returned in memory only.

P4 to P5 Upgrade Path

The only change required to use real AI in P5 is replacing the body of `_call_ai(prompt)` in `outreach.py` with a `boto3` invocation of AWS Bedrock (Claude Haiku). No database schema changes, no API contract changes, no frontend changes are needed.

Updated Development Specification — Story #7: Outreach Readiness Check

Dependent on Story #1 — Outreach Message Guidance

User Story

As a job seeker, I want feedback on my profile before messaging someone so that I present myself professionally.

GitHub Issue: <https://github.com/dhy-ani/linkedin-clone/issues/3>

Dependency on Story #1

Story #7 gates Story #1. The frontend calls GET /api/outreach/readiness before rendering the Outreach modal. If can_message is false, the readiness panel is shown in place of the modal. Both endpoints live in the same Flask process, use the same _auth_user() helper, and share the same users table. No new external dependencies are introduced.

Architecture Overview

The readiness check is a stateless scoring pass over the current user's profile. app.py receives the GET request, authenticates the caller, fetches the full user dict from database.py, and delegates scoring to compute_outreach_readiness() in outreach.py. That function evaluates the user dict against 9 weighted criteria using lambda functions and returns a scored result. No writes occur.

Sequence Diagram

Story #7 — Request Sequence

(Render at <https://mermaid.live> — paste block below)

```
sequenceDiagram
    participant Browser
    participant Flask as Flask app.py
    participant Outreach as outreach.py
    participant DB as database.py

    Browser->>Flask: GET /api/outreach/readiness
    Note over Flask: _auth_user() validates Bearer token
    Flask->>DB: get_current_user(user_id)
    DB-->>Flask: full user dict
    Flask->>Outreach: compute_outreach_readiness(user)
    Note over Outreach: Evaluates 9 weighted criteria via lambdas
    Outreach-->>Flask: {score, level, can_message, breakdown, top_tips}
    Flask-->>Browser: 200 JSON

    alt can_message == true
        Browser->>Flask: POST /api/outreach/generate (Story #1 flow)
    else can_message == false
        Browser->>Browser: Render readiness panel with top_tips
    end
```

Component Architecture Diagram

Story #7 — Component Architecture (classDiagram)

(Render at <https://mermaid.live> — paste block below)

```
classDiagram
    class AppModule {
        +outreach_readiness() GET /api/outreach/readiness
        +outreach_generate() POST /api/outreach/generate
        -_auth_user() dict
    }
    class OutreachModule {
        +compute_outreach_readiness(user) dict
        +generate_outreach_message(sender, recipient, context) dict
        -_OUTREACH_CHECKS list
        -_first_name(full_name) str
    }
    class DatabaseModule {
        +get_current_user(user_id) dict
        -_connect() connection
    }
    AppModule --> OutreachModule : calls compute_outreach_readiness
    AppModule --> DatabaseModule : calls get_current_user
    AppModule --> OutreachModule : calls generate_outreach_message (Story 1)
```

Unified Backend Data Flow

Stories #1 and #7 — Unified 4-Layer Data Flow

(Render at <https://mermaid.live> — paste block below)

```
flowchart TD
    subgraph Frontend
        FE[api.js Vanilla JS]
    end
    subgraph Backend
        GW[app.py Flask Gateway]
        BL[outreach.py Business Logic]
    end
    subgraph DataLayer
        DL[database.py]
        DB[(nexus.db SQLite WAL)]
    end
    subgraph AILayer
        AI[_call_ai]
        MOCK[P4: template passthrough]
        BEDROCK[P5: AWS Bedrock]
    end
    FE -->|POST /api/outreach/generate| GW
    FE -->|GET /api/outreach/readiness| GW
    GW --> BL
    GW --> DL
    DL --> DB
    BL --> AI
    AI -. ->|P4| MOCK
    AI -. ->|P5| BEDROCK
```

Technology Stack

Identical to Story #1. No additional dependencies are introduced.

Layer	Technology	Note
Frontend	HTML / CSS / Vanilla JS	Reads can_message field to gate Outreach modal
Backend	Python + Flask 3.0	Same Flask app instance as Story #1
Business Logic	outreach.py — compute_outreach_readiness()	Pure function, stateless, no I/O
Data Layer	database.py + SQLite WAL	get_current_user() read — no new writes

API

GET /api/outreach/readiness

No request body. Authorization: Bearer <token> header required.

Success response 200:

```
{ "score": 75, "max_score": 100, "level": "almost_ready", "can_message": true,
  "breakdown": [
    { "key": "headline", "label": "Headline set", "weight": 15, "met": true,
      "tip": "... " },
    ...
  ],
  "top_tips": ["Add at least one skill", "Upload a profile photo"] }
```

Error responses:

Code	Condition
401	No valid Bearer token
404	user_id from token not found in database

The 9 Readiness Criteria

Key	Label	Weight	Evaluation Logic
has_experience	Work experience listed	20	len(user.get('experience', [])) > 0
has_headline	Headline set	15	bool(user.get('headline', "").strip())
has_skills	Skills listed	15	len(user.get('skills', [])) > 0
has_photo	Profile photo set	10	user.get('avatarColor') is not None
has_about	About section filled	10	bool(user.get('about', "").strip())
has_education	Education listed	10	len(user.get('education', [])) > 0
has_connections	Has connections	10	user.get('connections', 0) >= 1
is_open_to_work	Open to Work enabled	5	bool(user.get('openToWork'))

has_website	Website or portfolio linked	5	bool(user.get('website', "").strip())
-------------	-----------------------------	---	---------------------------------------

Rep invariant: sum of all weights == 100. score == sum of weights for criteria where met == true.

Data Schemas

No new tables. Read-only access to the users table (same as Story #1):

- users.data JSON blob fields used: experience (list), headline (str), skills (list), avatarColor (any), about (str), education (list), connections (int), openToWork (bool), website (str).
- Missing keys are treated as falsy or empty — no KeyError is raised.

State Diagram — Readiness Check Flow

Story #7 — Readiness State

(Render at <https://mermaid.live> — paste block below)

```
stateDiagram-v2
    [*] --> CheckPending : User clicks Message button
    CheckPending --> CheckComplete : GET /api/outreach/readiness returns 200
    CheckPending --> CheckError : 401 or 404 or 500
    CheckError --> [*] : Show error toast
    CheckComplete --> OutreachOpen : can_message == true
    CheckComplete --> ReadinessPanel : can_message == false
    ReadinessPanel --> OutreachOpen : User clicks Proceed Anyway
    ReadinessPanel --> ProfileEdit : User clicks Improve Profile
    OutreachOpen --> [*] : Story 1 flow begins
```

Flow Chart — compute_outreach_readiness()

Story #7 — Backend Flow

(Render at <https://mermaid.live> — paste block below)

```
flowchart TD
    A[GET /api/outreach/readiness] --> B[_auth_user: validate Bearer token]
    B --> C{Token valid?}
    C -- No --> D[Return 401]
    C -- Yes --> E[get_current_user user_id]
    E --> F{User found?}
    F -- No --> G[Return 404]
    F -- Yes --> H[compute_outreach_readiness user]
    H --> I[Evaluate all 9 _OUTREACH_CHECKS lambdas]
    I --> J[sum weights for met == true]
    J --> K{score >= 70?}
    K -- Yes --> L[level = ready, can_message = true]
    K -- No --> M{score >= 40?}
    M -- Yes --> N[level = almost_ready, can_message = true]
    M -- No --> O[level = not_ready, can_message = false]
    L --> P[Select top_tips from unmet criteria]
    N --> P
    O --> P
    P --> Q[Return 200 JSON]
```

Security and Privacy

- Same auth model as Story #1: Bearer token required and validated via `_auth_user()` against `_sessions` dict and `sessions` table.
- `compute_outreach_readiness()` is purely stateless — it takes a dict, returns a dict, and performs no I/O. Cannot be used to exfiltrate data.
- The readiness score exposes profile completeness data about the requesting user only, never about other users.

Naming Consistency Note

Throughout this document the dependent story is referred to exclusively as Story #7 (Outreach Readiness Check). A reference to 'Story #2' in the P4 submission cover section was a typo; the correct identifier is Story #7, consistent with the GitHub issue tracker, all module-level naming in the codebase, and the original P2 development specification.